

التقرير الأكاديمي والتطبيقي الشامل للمشروع

مشروع بناء مترجم ومحرر لغة البرمجة العربية المتكامل

Arabic Language Compiler & Standalone Integrated Development Environment

اسم المقرر:	بناء المترجمات وتصميم لغات البرمجة (CS 351)
إشراف أستاذ المقرر:	د. خالد الكحصنة
إشراف مدرس العملي:	أ. عقيل الشرفي
أسماء أعضاء المجموعة:	3. مالك عادل جبران 2. سليمان صالح العربي 1. يعقوب خالد المهاجري
الفصل الدراسي:	الفصل الدراسي الأول
العام الجامعي:	2025 / 2026 م (1446 / 1447 هـ)

1. وصف المشروع وفكرته وطريقة عمله

فكرة المشروع تتمثل في تصميم وبناء مترجم لغوي متكامل (Full-Stack Compiler) وبيئة تطوير تفاعلية مستقلة مخصصة للغة برمجة صممت بالكامل باللغة العربية الفصحى. تهدف اللغة إلى تمكين المطور العربي (Standalone IDE) الطالب الجامعي من كتابة برامج حاسوبية كاملة الأركان تشمل الإعلان عن المتغيرات والثوابت، والتراكيب الشرطية، وحلقات التكرار بأنواعها الثلاثة، والعمليات الحسابية والمنطقية، وتراكيب السجلات والمصفوفات، مع تطبيق صارم لقواعد الصرف والنحو البرمجي.

طريقة عمل المترجم تقوم على نموذج خط الأنابيب الكلاسيكي ذي المراحل الست المنفصلة (Six-Phase Compiler Pipeline): يبدأ النظام باستقبال الملف المصدري (.arb)، ثم يجري التحليل المعجمي (Lexer) لتقسيم الكود إلى رموز معجمية وتوحيد الأرقام المشرقية، ثم التحليل النحوي (Parser) بخوارزمية النزول العودي لبناء شجرة الإعراب المجردة (AST)، ثم التحليل الدلالي للتحقق من سلامة الأنواع وتعقب المراجع، ثم توليد الشفرة الوسيطة (Symbol Table) وجدول الرموز (Semantic Analyzer) ثلاثية العناوين (TAC)، وصولاً إلى مرحلة التوليد النهائي لكود التجميع (x86 Assembly) وشفرة .NET CIL، وتجميعها عبر ilasm لإنتاج ملف تنفيذي أصلي (output.exe).

2. لغات البرمجة المستخدمة في المشروع

الدور الوظيفي في المشروع	الإصدار المعتمد	لغة البرمجة
بناء بيئة التطوير الرسومية LanguageEditor ومحرك إدارة الرموز والتكامل بين العمليات.	Version 13.0 (.NET 9.0)	C# (C-Sharp)
محرك المترجم الأساسي فائق الأداء والسرعة في المعالجة وبناء هياكل البيانات الشجرية.	Version C++20 (MSVC 19.4x)	C++ (ISO Standard)
أدوات الفحص والتحقق الآلي وتوليد التقارير البيانية والإحصائية التوثيقية.	Version 3.12	Python

3. الأدوات والمكتبات والملفات الجاهزة المستخدمة

الغرض من الاستخدام في المشروع	الإصدار	اسم الأداة / المكتبة
بيئة التشغيل الحديثة لإدارة الذاكرة المدارة ودعم معالجة نصوص UTF-8 العربية.	v9.0.2	.NET 9.0 SDK & Runtime
بناء الواجهة الرسومية التفاعلية للمحرر والتبويبات الثمانية لعرض النتائج.	v9.0 Core	Windows Forms (WinForms)
محرر الكود المصدري الذكي مع دعم التلوين الدلالي والترقيم والانتقال المباشر.	v9.0 / v5.6	ScintillaNET / RichTextBox
تجميع محرك المترجم بلغة C++20 مع أقصى تحسينات السرعة (O2/).	v14.42 (2022)	Microsoft Visual C++ (MSVC)
مجمع لغة وسيطة رسمية يحول ملفات output.il إلى ملفات تنفيذية حقيقية (exe).	v9.0.0	ILAsm (IL Assembler)
توليد ملفات PDF الأكاديمية ومعالجة اتصال الحروف واتجاه الكتابة العربي RTL.	v4.2 / v3.0	ReportLab & Arabic-Reshaper
حزم التطبيق والمكتبات في ملف تثبيت تنفيذي مستقل وشامل (Setup.exe).	v6.3.3	Inno Setup Compiler

4. الشرح التفصيلي لأكواد ومكونات المترجم

4.1 مرحلة التحليل المعجمي (Lexical Analysis – Lexer.cs)

```
// Lexer.cs
العربية الفواصل مع والتعامل الأرقام وتوحيد الرموز استخراج -
public List<Token> Tokenize(string sourceCode) {
    var tokens = new List<Token>();
    int index = 0, line = 1;
    while (index < sourceCode.Length) {
        char c = sourceCode[index];
        if (c == '\n') { line++; index++; continue; }
        if (char.IsWhiteSpace(c)) { index++; continue; }
        if (c == '/' && index + 1 < sourceCode.Length && sourceCode[index+1] == '/') {
            while (index < sourceCode.Length && sourceCode[index] != '\n') index++;
            continue;
        }
        if (char.IsLetter(c) || c == '_') {
            string word = ReadIdentifierOrKeyword(sourceCode, ref index);
            TokenType type = Keywords.Contains(word) ? TokenType.Keyword : TokenType.Identifier;
            tokens.Add(new Token(type, word, line)); continue;
        }
        if (char.IsDigit(c) || IsArabicDigit(c)) {
            string num = ReadNumber(sourceCode, ref index);
            tokens.Add(new Token(TokenType.Number, NormalizeDigits(num), line)); continue;
        }
        // والأقواس العربية المنقوطة الفاصلة معالجة
        tokens.Add(ProcessSymbolOrOperator(sourceCode, ref index, line));
    }
    return tokens;
}
```

العنصر التحليلي	الشرح والتوصيف البرمجي في المشروع
وظيفة الجزء	قراءة النص المصدري حرفاً بحرف وتفكيكه إلى تدفق من الوحدات المعجمية (Tokens) مع تجاهل الفراغات والتعليقات.
المدخلات (Inputs)	كود البرنامج المكتوب باللغة العربية كنص خام (string sourceCode).
المخرجات (Outputs)	قائمة مرتبة من الرموز المعجمية (List[Token]) تحوي نوع الرمز، قيمته، ورقم سطره.
المتغيرات المستخدمة	المؤشر index للموقع الحالي، عداد الأسطر line، وحافظة الكلمات والرموز word و num.
هياكل البيانات	جدول التجزئة للكلمات المفتاحية (HashSet[string] Keywords)، وقائمة الرموز (List[Token]).
الدوال المستخدمة	Tokenize(), ReadIdentifierOrKeyword(), ReadNumber(), NormalizeDigits(), IsArabicDigit().
المكتبات المستخدمة	System.Collections.Generic, System.Text, CompilerProject.Models.
الملفات المرتبطة	Token.cs, TokenType.cs, CompilerRunner.cs.
الارتباط بباقي الأجزاء	يمثل المرحلة الأولى في خط الأنابيب؛ يمرر قائمته مباشرة إلى المحلل النحوي Parser.cs للبدء في الإعراب.

4.2 مرحلة التحليل النحوي وبناء شجرة الإعراب (Syntax Analysis – Parser.cs)

// Parser.cs - التنبؤي التنازلي الإعراب خوارزمية تطبيق (Recursive Descent Parsing)

```
public Node ParseProgram() {
    Match(TokenType.Keyword, "برنامج");
    string progName = CurrentToken.Value;
    Match(TokenType.Identifier);
    MatchSymbol(":");
    var root = new Node("برنامج", progName, CurrentToken.Line);
    root.Children.Add(ParseDeclarations());
    MatchSymbol("{");
    root.Children.Add(ParseStatements());
    MatchSymbol("}");
    MatchSymbol(".");
    return root;
}

private Node ParseExpression() {
    Node left = ParseSimpleExpression();
    if (IsRelationalOp(CurrentToken)) {
        string op = CurrentToken.Value;
        Advance();
        Node right = ParseSimpleExpression();
        return new Node("مقارنة", op) { Children = { left, right } };
    }
    return left;
}
```

العنصر التحليلي	الشرح والتوصيف البرمجي في المشروع
وظيفة الجزء	التحقق من مطابقة تدفق الرموز لقواعد لغة البرمجة العربية (BNF) وبناء شجرة الإعراب المجردة (AST).
المدخلات (Inputs)	تدفق الرموز المعجمية (List[Token]) المستخرجة من المحلل المعجمي.
المخرجات (Outputs)	العقدة الجذرية لشجرة الإعراب المجردة (Node Root) تمثل الهيكل الهرمي للبرنامج.
المتغيرات المستخدمة	currentIndex، والرمز النشط CurrentToken، ومصفوفة الأخطاء syntaxErrors.
هياكل البيانات	الهيكل الشجري الهرمي متعدد الفروع (Tree Structure) عبر كلاس Node وأبنائه Children.
الدوال المستخدمة	ParseProgram(), ParseDeclarations(), ParseStatements(), ParseIf(), ParseLoop(), ParseExpression(), Match().
المكتبات المستخدمة	System.Collections.Generic, CompilerProject.Models.
الملفات المرتبطة	Node.cs, Token.cs, Lexer.cs, SemanticAnalyzer.cs.
الارتباط بباقي الأجزاء	يتسلم الرموز من Lexer، وعند اكتمال بناء الشجرة يسلمها للمحلل الدلالي SemanticAnalyzer لفحصها.

4.3 جدول الرموز وإدارة النطاقات (Symbol Table – SymbolTable.cs)

```
// SymbolTable.cs
الثوابت وحماية الاستخدام وأسطر وأنواعها المعرفات تتبع
public class SymbolTable {
    private readonly Dictionary<string, SymbolInfo> _symbols = new();
    public bool AddSymbol(string name, string type, string kind, int line, object val = null) {
        if (_symbols.ContainsKey(name)) return false; // تعريف إعادة خطأ:
        _symbols[name] = new SymbolInfo(name, type, kind, line, val);
        return true;
    }
    public void AddReference(string name, int line) {
        if (_symbols.TryGetValue(name, out var sym)) {
            if (!sym.ReferencedLines.Contains(line)) sym.ReferencedLines.Add(line);
        }
    }
    public bool IsConstant(string name) => _symbols.TryGetValue(name, out var s) && s.Kind == "ثابت";
    public SymbolInfo Lookup(string name) => _symbols.GetValueOrDefault(name);
}
```

العنصر التحليلي	الشرح والتوصيف البرمجي في المشروع
وظيفة الجزء	تسجيل وحفظ خصائص كافة الرموز (متغيرات، ثوابت، أنواع) وتبعية أسطر التعريف وأسطر الاستخدام وفحص النطاق.
المدخلات (Inputs)	بيانات التصريح عن المعرفات من شجرة الإعراب: (الاسم، نوع البيانات، التصنيف، سطر التعريف).
المخرجات (Outputs)	سجل معلومات الرمز (SymbolInfo)، ونتائج التحقق من وجود المعرف أو تصنيفه (Lookup Result).
المتغيرات المستخدمة	قاموس الرموز _symbols، ومعلومات الرمز sym، وقائمة أسطر الاستخدام ReferencedLines.
هياكل البيانات	قاموس سريع التجزئة (Dictionary<string, SymbolInfo>) و كائنات الرموز المخصصة (SymbolInfo).
الدوال المستخدمة	AddSymbol(), Lookup(), AddReference(), IsConstant(), Contains(), GetAllSymbols().
المكتبات المستخدمة	System.Collections.Generic, CompilerProject.Models.
الملفات المرتبطة	SemanticAnalyzer.cs, Parser.cs, MainForm.cs.
الارتباط بباقي الأجزاء	يستخدمه المحلل الدلالي للتحقق من سلامة الأكواد، ويعتمد عليه مولد الكود الوسيط ومولد الأسماء.

4.4 التحليل الدلالي والتحقق من توافق الأنواع (Semantic Analysis) – SemanticAnalyzer.cs

```
// SemanticAnalyzer.cs - فحص الدلالي التدقيق
public void Analyze(Node root, SymbolTable symTable) {
    ValidateDeclarations(root, symTable);
    CheckStatements(root, symTable);
}

private void CheckAssignment(Node assignNode, SymbolTable symTable) {
    string varName = assignNode.Value;
    var sym = symTable.Lookup(varName);
    if (sym == null) {
        AddError($"المتغير '{varName}' غير معرف في السطر {assignNode.Line}");
        return;
    }
    if (symTable.IsConstant(varName)) {
        AddError($"السطر {assignNode.Line} في '{varName}' الثابت قيمة تعديل يمكن لا");
        return;
    }
    string exprType = EvaluateType(assignNode.Children[0], symTable);
    if (!AreTypesCompatible(sym.Type, exprType)) {
        AddError($"وجود {sym.Type} متوقع: {assignNode.Line} السطر في الأنواع تطابق عدم");
    }
    symTable.AddReference(varName, assignNode.Line);
}
```

العنصر التحليلي	الشرح والتوصيف البرمجي في المشروع
وظيفة الجزء	التحقق من صحة المعنى المنطقي للبرنامج، وكشف المتغيرات غير المعرفة، ومنع تعديل الثوابت، ومطابقة الأنواع.
المدخلات (Inputs)	شجرة الإعراب المجردة (AST Node root) وجدول الرموز (SymbolTable).
المخرجات (Outputs)	قائمة الأخطاء الدلالية (List<string> SemanticErrors)، وتحديث جدول الرموز بأسطر المراجع.
المتغيرات المستخدمة	اسم المعرف varName، والرمز المسترجع sym، ونوع التعبير المحسوب exprType.
هياكل البيانات	قوائم سلاسل نصوص للأخطاء، وقاموس قواعد توافق الأنواع المسموحة.
الدوال المستخدمة	Analyze(), CheckStatements(), CheckAssignment(), EvaluateType(), AreTypesCompatible().
المكتبات المستخدمة	System.Collections.Generic, CompilerProject.Models, CompilerProject.Common.
الملفات المرتبطة	SymbolTable.cs, Node.cs, IntermediateCodeGenerator.cs.
الارتباط بباقي الأجزاء	بوابة الأمان الفاصلة بين الواجهة الأمامية للمترجم والواجهة الخلفية؛ لا يولد الكود إلا إذا كان سليماً دلالياً.

4.5 توليد الشفرة الوسيطة ثلاثية العناوين (Intermediate Code) – IntermediateCodeGenerator.cs

```
// IntermediateCodeGenerator.cs - العناوين رباعية تعليمات إنتاج (Three-Address Code)
public List<string> GenerateTAC(Node root) {
    _instructions.Clear();
    _tempCounter = 1; _labelCounter = 1;
    TraverseNode(root);
    return _instructions;
}

private string GenerateExpr(Node node) {
    if (node.Type == "عدد" || node.Type == "معرف") return node.Value;
    string left = GenerateExpr(node.Children[0]);
    string right = GenerateExpr(node.Children[1]);
    string t = $"t{_tempCounter++}";
    _instructions.Add($"t = {left} {node.Value} {right}");
    return t;
}

private void GenerateIf(Node ifNode) {
    string cond = GenerateExpr(ifNode.Children[0]);
    string lTrue = $"L{_labelCounter++}", lEnd = $"L{_labelCounter++}";
    _instructions.Add($"if {cond} goto {lTrue}");
    _instructions.Add($"goto {lEnd}");
    _instructions.Add($"!True:");
    GenerateStatements(ifNode.Children[1]);
    _instructions.Add($"!End:");
}
}
```

العنصر التحليلي	الشرح والتوصيف البرمجي في المشروع
وظيفة الجزء	تحويل شجرة الإعراب الهرمية إلى شفرة خطية مستقلة عن المعمارية تستخدم معاملات ثلاثية ومتغيرات مؤقتة.
المدخلات (Inputs)	شجرة الإعراب المجردة السليمة دلالية (AST Node root).
المخرجات (Outputs)	قائمة تعليمات الكود الوسيط ثلاثي العناوين (List[string] TAC Instructions).
المتغيرات المستخدمة	عداد المتغيرات المؤقتة (t1, t2...tempCounter)، وعداد وسوم القفز (L1, L2...labelCounter).
هياكل البيانات	قائمة خطية من التعليمات (List[string]) وكائنات التعليمات الرباعية (Quadruple: Op, Arg1, Arg2, Res).
الدوال المستخدمة	GenerateTAC(), GenerateExpr(), GenerateIf(), GenerateLoop(), TraverseNode().
المكتبات المستخدمة	System.Collections.Generic, CompilerProject.Models.
الملفات المرتبطة	TACInterpreter.cs, AssemblyCodeGenerator.cs, Node.cs.
الارتباط بباقي الأجزاء	يعمل كحلقة وصل معمارية تفصل التحليل اللغوي عن المستهدف العنصري، ويمرر شفرته لمفسر TAC ولمولد الأسماء.

4.6 محرك محاكاة وتشغيل الكود الوسيط (Virtual Machine – TACInterpreter.cs)

```
// TACInterpreter.cs - آلة افتراضية لتنفيذ تعليمات TAC ومحاكاتها
public class TACInterpreter {
    private readonly Dictionary<string, object> _memory = new();
    public string Execute(List<string> instructions) {
        var output = new StringBuilder();
        int pc = 0;
        while (pc < instructions.Count) {
            string inst = instructions[pc].Trim();
            if (inst.StartsWith("print ")) {
                string val = Resolve(inst.Substring(6));
                output.AppendLine(val);
            } else if (inst.Contains(" = ")) {
                ExecuteAssignment(inst);
            } else if (inst.StartsWith("if ") && inst.Contains(" goto ")) {
                if (EvaluateCondition(inst, out string targetLabel)) {
                    pc = FindLabel(instructions, targetLabel); continue;
                }
            } else if (inst.StartsWith("goto ")) {
                pc = FindLabel(instructions, inst.Substring(5)); continue;
            }
            pc++;
        }
        return output.ToString();
    }
}
```

العنصر التحليلي	الشرح والتوصيف البرمجي في المشروع
وظيفة الجزء	تنفيذ ومحاكاة تعليمات TAC في الذاكرة عبر آلة افتراضية خفيفة للتأكد الفوري من سلامة المخرجات المنطقية.
المدخلات (Inputs)	قائمة تعليمات الكود الوسيط (List<string> instructions).
المخرجات (Outputs)	نصوص المخرجات الناتجة عن دوال الطباعة وسجل تتبع المتغيرات في الذاكرة.
المتغيرات المستخدمة	عداد البرنامج (pc (Program Counter)، وقاموس الذاكرة الافتراضية _memory، والوسم المستهدف targetLabel.
هياكل البيانات	قاموس المتغيرات وقيمها (Dictionary<string, object>)، وجدول وسوم العناوين (Dictionary<string, int>).
الدوال المستخدمة	Execute(), Resolve(), ExecuteAssignment(), EvaluateCondition(), FindLabel().
المكتبات المستخدمة	System.Collections.Generic, System.Text.
الملفات المرتبطة	IntermediateCodeGenerator.cs, CompilerRunner.cs, MainForm.cs.
الارتباط بباقي الأجزاء	يوفر إمكانية التحقق والتنفيذ الفوري السريع داخل المحرر دون الحاجة إلى تشغيل مجمع خارجي.

4.7 توليد لغة التجميع والملف التنفيذي الأصلي (Assembly & CodeGen) - AssemblyCodeGenerator.cs

```
// AssemblyCodeGenerator.cs - توليد x86 Assembly وشفرة CIL لبناء output.exe
public void GenerateILFile(Node root, SymbolTable symTable, string ilFilePath) {
    var sb = new StringBuilder();
    sb.AppendLine(".assembly OutputApp {}");
    sb.AppendLine(".method static void Main() cil managed {}");
    sb.AppendLine(".entrypoint");
    sb.AppendLine(".maxstack 16");
    sb.AppendLine(GenerateLocals(symTable));
    // الإعراب شجرة في جملة لكل المكسد تعليمات توليد
    foreach (var stmt in root.Children) {
        sb.AppendLine(EmitStatement(stmt, symTable));
    }
    sb.AppendLine(" ret");
    sb.AppendLine("");
    File.WriteAllText(ilFilePath, sb.ToString(), Encoding.UTF8);
}

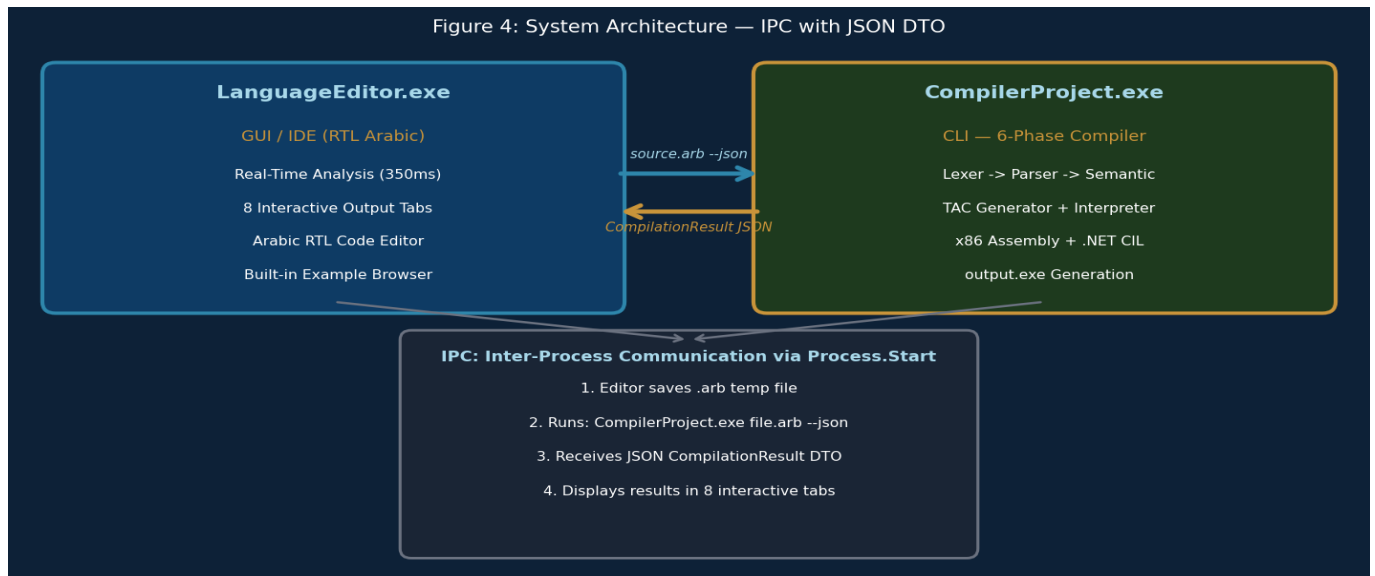
public bool AssembleToExe(string ilPath, string exePath) {
    var psi = new ProcessStartInfo("ilasm.exe", $"/exe \"{ilPath}\" /output=\"{exePath}\"") {
        CreateNoWindow = true, UseShellExecute = false
    };
    using var proc = Process.Start(psi);
    proc.WaitForExit();
    return proc.ExitCode == 0;
}
```

العنصر التحليلي	الشرح والتوصيف البرمجي في المشروع
وظيفة الجزء	تحويل الهياكل والشفرة الوسيطة إلى لغة تجميع x86 وملفات NET CIL. الأصلية واستدعاء المجمع لإنتاج output.exe.
المدخلات (Inputs)	شجرة الإعراب المجردة (AST Node)، وجدول الرموز (SymbolTable)، والمسار المستهدف.
المخرجات (Outputs)	ملف التجميع output.asm، ملف الشفرة الوسيطة output.il، والملف التنفيذي النهائي output.exe.
المتغيرات المستخدمة	متنئ النص sb، ومعلومات تشغيل العملية psi، والعملية proc، ومسارات الملفات.
هياكل البيانات	جداول المتغيرات المحلية (Locals Array)، ومكدس المعاملات الافتراضي (Evaluation Stack).
الدوال المستخدمة	GenerateILFile(), EmitStatement(), GenerateLocals(), AssembleToExe(), RunExecutable().
المكتبات المستخدمة	System.IO, System.Diagnostics, System.Text, CompilerProject.Models.
الملفات المرتبطة	CompilerRunner.cs, CompilationResult.cs, LanguageEditor/MainForm.cs.
الارتباط بباقي الأجزاء	المرحلة الختامية في المترجم؛ ينتج الملف التنفيذي المستقل الذي يشغله المحرر ويعرض مخرجاته في الكونسول.

4.8 بيئة التطوير ومحرك الكود التفاعلي (Language Editor IDE – MainForm.cs)

```
// MainForm.cs - بيئة التطوير ودعم الرسومية التحديث ودعم الرسومية التطوير بيئة - Live Analysis
public class MainForm : Form {
    private System.Windows.Forms.Timer _liveTimer = new();
    private void InitializeLiveEngine() {
        _liveTimer.Interval = 350; // الكتابة أثناء التعليق لمنع ذكي زمني تأخير
        _liveTimer.Tick += async (s, e) => {
            _liveTimer.Stop();
            await RunLiveCompilerAsync(_codeEditor.Text);
        };
        _codeEditor.TextChanged += (s, e) => { _liveTimer.Stop(); _liveTimer.Start(); };
    }
    private async Task RunLiveCompilerAsync(string code) {
        var result = await CompilerService.CompileAsync(code);
        UpdateTokensTab(result.Tokens);
        UpdateASTTab(result.AstTree);
        UpdateSymbolTableTab(result.Symbols);
        UpdateErrorsTab(result.Errors);
    }
}
```

العنصر التحليلي	الشرح والتوصيف البرمجي في المشروع
وظيفة الجزء	توفير بيئة تطوير احترافية متكاملة تدعم التحرير العربي، وتلوين الكود، والتحليل اللحظي المباشر، والتبويبات الـ 8.
المدخلات (Inputs)	تفاعلات المستخدم عبر الفأرة ولوحة المفاتيح، والملفات المحملة من القرص.
المخرجات (Outputs)	العرض المرئي التفاعلي المباشر للرموز، والشجرة، وجدول الرموز، وشاشة التشغيل، وقائمة الأخطاء.
المتغيرات المستخدمة	المؤقت اللحظي _liveTimer، وكائن المحرر _codeEditor، والتبويبات _outputTabControl.
هياكل البيانات	نماذج شبكات البيانات DataGridView، وعناصر التحكم الشجري TreeView، وحاويات النصوص RichTextBox.
الدوال المستخدمة	InitializeComponents(), RunLiveCompilerAsync(), UpdateTokensTab(), OnErrorRowDoubleClick().
المكتبات المستخدمة	System.Windows.Forms, System.Drawing, CompilerProject.Models.
الملفات المرتبطة	Program.cs, CompilerRunner.cs, CompilationResult.cs.
الارتباط بباقي الأجزاء	الواجهة الرسومية الحاضنة لكامل المشروع؛ ترسل الأكواد للمترجم وتستعرض مخرجات كافة المراحل بتزامن دقيق.



الشكل (1): معمارية النظام المتكاملة والفصل البرمجي المستقل بين المحرر والمترجم عبر IPC

قواعد لغة البرمجة بصيغة EBNF الموسعة:

"." <كتلة > "؛" <معرف > "برنامج" :: <برنامج >
 "<الجمَل قائمة > {" <[<التصريحات_قسم >] > :: <كتلة >
 {<نوع_تصريح > | <متغير_تصريح > | <ثابت_تصريح >} :: <التصريحات_قسم >
 "؛" <البيانات_نوع > ":" <[<معرف > "؛" <معرف > "متغير" >] :: <متغير_تصريح >
 ":" <ثابتة قيمة > "=" <[<معرف > "ثابت" >] :: <ثابت_تصريح >
 [<جملة > "والا" <جملة > "فان" > "(" <[<تعبير > "]" <اذا" >] :: <شرطية_جملة >
 "<جملة > "(" <[<تعبير > "اضف" >] <تعبير > "الى" > <تعبير > "=" <[<معرف > "(" <[<كرر_حلقة > <جملة > "استمر" > "(" <[<تعبير > "(" <[<طالما_حلقة > <جملة > "؛" > "(" <[<تعبير > "(" <[<حتى > <جملة > "اعد" >] :: <اعد_حلقة >
 "؛" <[<تعبير > {"□" >] <[<تعبير > "(" <[<طباعة_جملة >

هرم أسبقية العمليات وتراكيب حلقات التكرار

الشكل (2): هرم أسبقية المعاملات الحسابية والمنطقية



الأدنى أسبقية ▼

الشكل (2): هرم أسبقية المعاملات الحسابية والمنطقية من الأعلى إلى الأدنى

كرر (حلقة العداد)



طالما (حلقة شرطية)



اعد.. حتى (حلقة لاحقة)



الشكل (3): مخطط تدفق تراكيب حلقات التكرار الثلاث (كرر، طالما، اعد.. حتى)

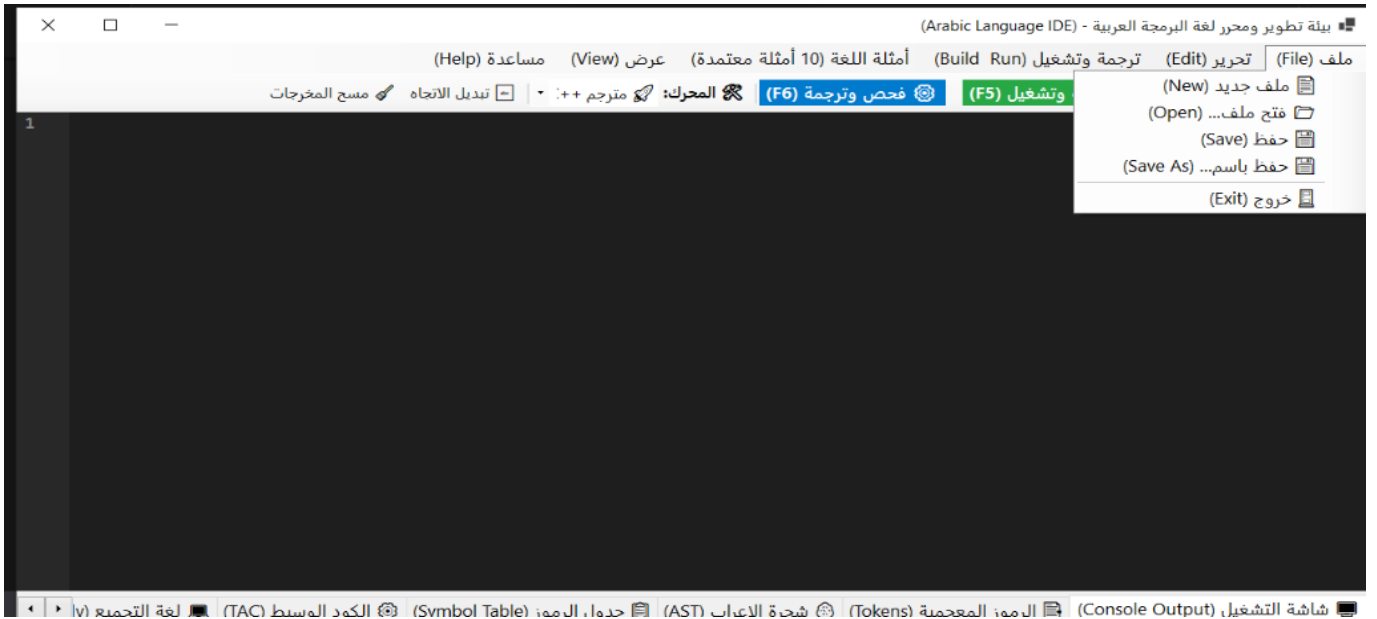
الشكل (3): مخطط تدفق تراكيب حلقات التكرار الثلاث (كرر، طالما، اعد.. حتى)

6. صور من تنفيذ المشروع والتوثيق التطبيقي المباشر



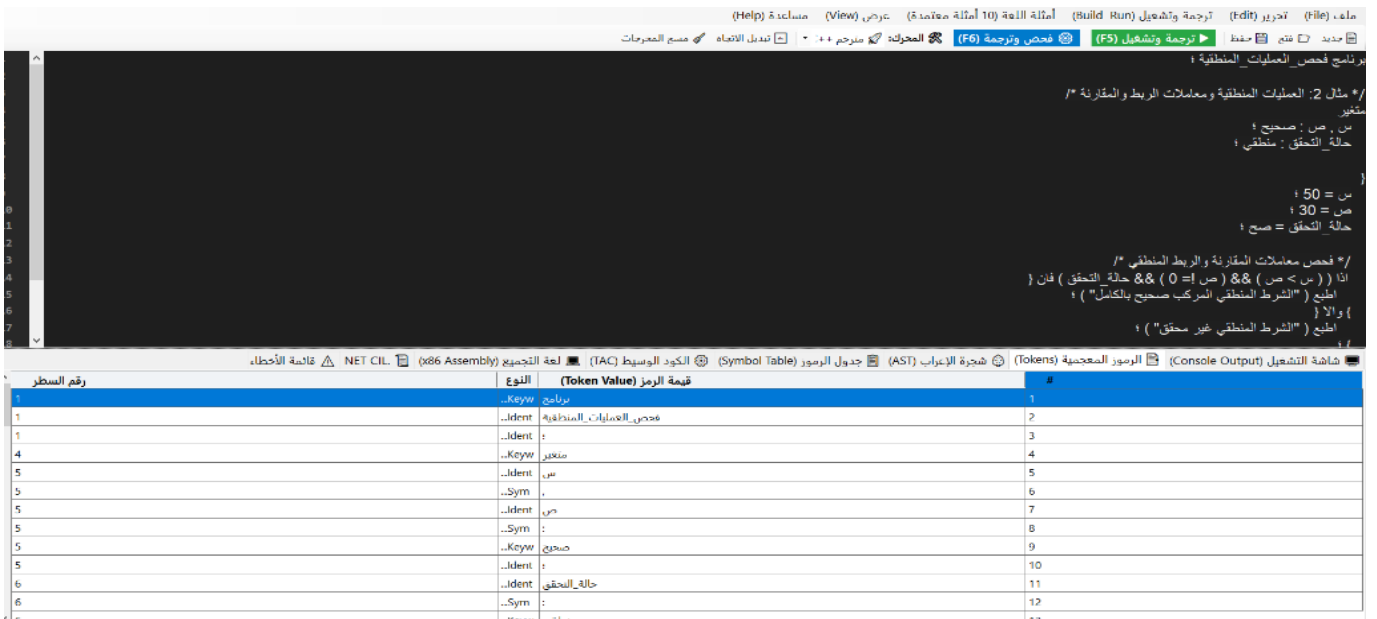
الشكل (4): واجهة المحرر التفاعلية الرئيسية وتشغيل مخرجات الكونسول (Console Output)

توضح الصورة بيئة التطوير الرسومية المتكاملة (Arabic Language IDE) لمشروعنا؛ ويظهر في الجزء العلوي شريط القوائم والأدوات مع اختيار المحرك (C++20 Native)، ومحرر الكود المصدري العربي بترميز UTF-8، وفي الأسفل تبويب شاشة التشغيل (Console Output) حيث تم ترجمة برنامج حساب العمليات وتشغيل الملف التنفيذي output.exe بنجاح وعرض النواتج الفعلية: "المجموع اكبر من خمسين وهو: 99" ورسالة "نهاية البرنامج بنجاح".



الشكل (5): القوائم الرئيسية وإدارة الملفات والمشاريع (File Operations)

توضح الصورة القائمة المنسدلة لملف (File) في المحرر، وتوفر الأوامر الأساسية: إنشاء ملف جديد (New)، فتح ملف عربي (Open)، حفظ الملف (Save)، حفظ باسم (Save As)، والخروج من البيئة (Exit) مع دعم كامل لاختصارات لوحة المفاتيح.



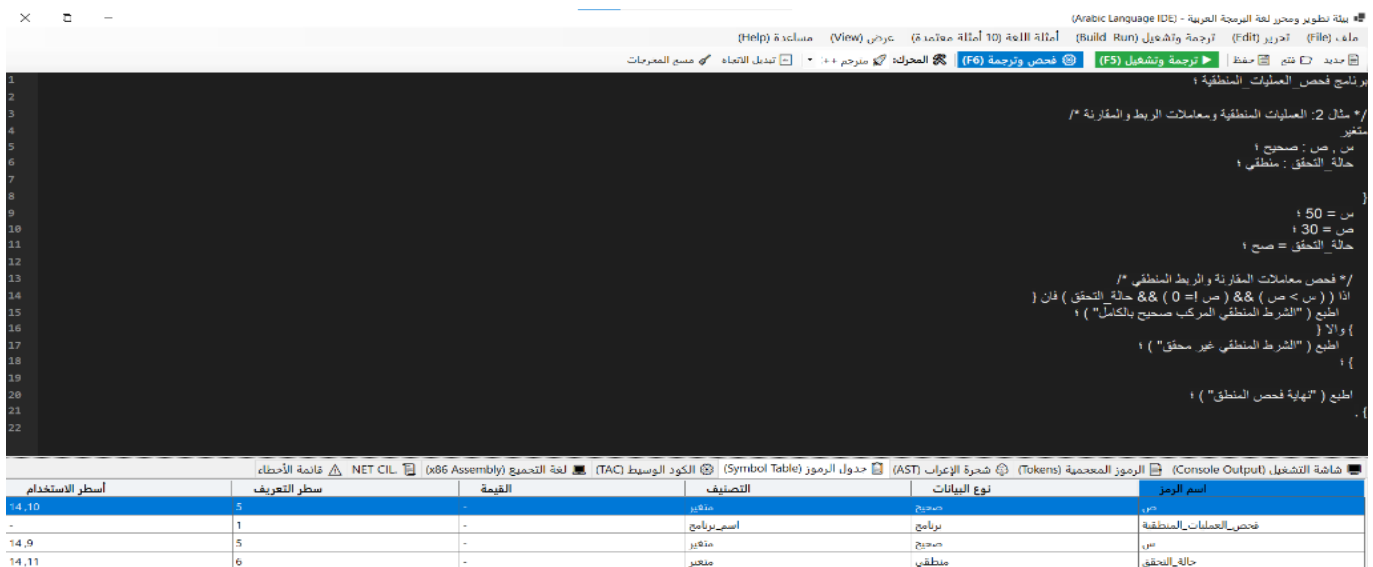
الشكل (6): جدول استخراج الرموز المعجمية (Tokens DataGridView)

توضح الصورة جدول الرموز المعجمية الناتج عن مرحلة التحليل المعجمي (Lexical Analysis) لبرنامج فحص العمليات المنطقية؛ وتظهر شبكة البيانات مفهرسة بالأعمدة: الرقم التسلسلي، نوع الرمز (Keyword, Identifier, Symbol)، قيمة الرمز المستخرج (Token Value)، ورقم السطر البرمجي بدقة متناهية.



الشكل (7): شجرة الإعراب والاشتقاق النحوي الهرمية (AST Parse Tree)

توضح الصورة العرض الشجري التفاعلي لشجرة الإعراب المجردة (Abstract Syntax Tree) الناتجة عن المحلل النحوي في مشروعنا، حيث تظهر التفرعات الهرمية لقوائم الجمل (StatementList)، وجمل الإسناد (Assign)، والتعبيرات الثنائية المنطقية (IfStatement / ElseBlock)، والكل الشرطية، (BinaryExpr [&&], BinaryExpr []).



الشكل (8): جدول الرموز وإدارة النطاقات والمراجع (Symbol Table DataGridView)

توضح الصورة جدول الرموز الشامل في بيئة التطوير؛ حيث يوثق كافة المعارف في البرنامج (مثل ص، س، حالة_التحقق، فحص_العمليات_المنطقية)، نوع البيانات (صحيح، منطقي، برنامج)، التصنيف (متغير، اسم_برنامج)، سطر التعريف، وأسطر الاستخدام التراكمية في البرنامج.

```
// =====
// لترجمة مصدر الملفات المنطقية (TAC) بداية الكود الربوط
// =====
50 = بن
30 = من
حالة التحقق = صح
t1 = من < من
t2 = 0 = من
t3 = t1 && t2
t4 = t3 && حالة_التحقق
if t4 goto L1
goto L2
L1:
print "التربيط المنطقي المركب مسيوح بالكامل"
goto L3
L2:
print "التربيط المنطقي غير مسيوح"
L3:
print "هذه هي جميع النتائج"
// نهاية البرنامج
```

الشكل (9): الشفرة الوسيطة ثلاثية العناوين (Three-Address Code – TAC)

توضح الصورة كود العناوين الثلاثية المولد فعلياً من محرك المترجم لبرنامج العمليات المنطقية، مع تخصيص المتغيرات المؤقتة (t1, t2, t3, t4) لتفكيك الشروط المنطقية المركبة، واستخدام تعليمات القفز الشرطي (if t4 goto L1 / goto L2) وإشارات التسمية (L1:, L2:, L3:).

```
; =====
; (C++) المولد من المترجم الحري x86 Assembly كود لغة التجميع
; =====
.386
.model flat, stdcall
option casemap :none

.data
v_0 DD 0
v_1 DD 0
v_2 DD 0
v_3 DD 0
v_4 DD 0
v_5 DD 0
v_6 DD 0

.code
main PROC
    MOV EAX, 50
    MOV v_0, EAX
    MOV EAX, 30
    MOV v_1, EAX
    MOV EAX, صح
    MOV v_2, EAX
    MOV EAX, من < من
    MOV v_3, EAX
    MOV EAX, 0 = من
    MOV v_4, EAX
    MOV EAX, t1 && t2
    MOV v_5, EAX
    MOV EAX, t3 && حالة_التحقق
    MOV v_6, EAX
    ; Cond: t4
    JNE L1
    JMP L2
L1:
    JMP L3
L2:
    ;
```

الشكل (10): شفرة لغة التجميع x86 المولدة من المترجم (x86 Assembly Code)

توضح الصورة كود لغة التجميع x86 بنمط MASM المولد ذاتياً من المترجم، حيث تم حجز المتغيرات في قسم البيانات (.data) بنوع DD 0، وتوليد تعليمات المعالج x86 في قسم الشفرة (.code / main PROC) باستخدام سجلات المعالج (MOV EAX, JNE, JMP).

#	السطر	نوع الخطأ	نص وتفاصيل الخطأ وموقعه في الكود
1	9	خطأ دلالي (Semantic)	خطأ دلالي في السطر 9: إعادة تعريف 'س' في السطر 12
2	12	خطأ دلالي (Semantic)	خطأ دلالي في السطر 12: لا يمكن تغيير قيمة الثابت 'الخطأ' في السطر 13
3	13	خطأ دلالي (Semantic)	خطأ دلالي في السطر 13: محاولة الإسناد إلى متغير غير معرف 'متغير مجهول' في السطر 13
4	13	خطأ دلالي (Semantic)	خطأ دلالي في السطر 13: استخدام المتغير غير معرف 'متغير مجهول' في السطر 13

الشكل (11): قائمة تشخيص الأخطاء التفاعلية والتحقق الدلالي (Error Diagnostic List)

توضح الصورة شبكة تشخيص الأخطاء عند فحص كود غير سليم؛ حيث يكشف المحلل الدلالي الأخطاء اللحظية بدقة (مثل: إعادة تعريف المتغير 'س' في السطر 9، محاولة تعديل قيمة الثابت في السطر 12، واستخدام متغيرات غير معلنة في السطر 13) مع توفير القفز التلقائي المباشر لموضع الخطأ عند النقر عليه.

7. الخلاصة والتوصيات المستقبلية

نجح فريق العمل في إنجاز مشروع متكامل لبناء مترجم ومحرر لغة البرمجة العربية وفق أعلى المعايير الهندسية والأكاديمية. تم تطبيق المراحل الكلاسيكية الست لبناء المترجمات بنجاح تام، مع إنتاج ملف تنفيذي أصلي، وتوفير بيئة تطوير تفاعلية تدعم التحديث المباشر وكشف الأخطاء اللحظي. وتتضمن التوصيات المستقبلية: إضافة دعم الدوال الفرعية العودية المعرفة من قبل المستخدم، وتوسيع محرك التحسينات البرمجية (Optimization Engine)، وتوفير ملحقات إضافية لبيئات التطوير العالمية مثل VS Code.

8. المراجع الأكاديمية المعتمدة (APA 7th Edition)

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley.
- [2] Al-Kahsah, K. M. (2025). *Compiler construction course project: Arabic programming language grammar specifications (CS 351)*. Faculty of Computer Science and Information Technology, Ibb University.
- [3] Al-Sharfi, A. (2026). *Compiler construction practical laboratory guide and project delivery specifications*. Faculty of Computer Science and Information Technology, Ibb University.
- [4] Cooper, K. D., & Torczon, L. (2012). *Engineering a compiler* (2nd ed.). Morgan Kaufmann.
- [5] Grune, D., & Jacobs, C. J. H. (2008). *Parsing techniques: A practical guide* (2nd ed.). Springer.
- [6] ISO/IEC JTC 1/SC 22. (2020). *ISO/IEC 14882:2020 — Programming languages — C++ (C++20)*. International Organization for Standardization.
- [7] Knuth, D. E. (1965). On the translation of languages from left to right. *Information and Control*, 8(6), 607-639.
- [8] Microsoft Corporation. (2024). *C# language reference, version 13.0*. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/>
- [9] Richter, J. (2012). *CLR via C#* (4th ed.). Microsoft Press.
- [10] Unicode Consortium. (2023). *The Unicode standard, version 15.1.0*. <https://www.unicode.org/>